Swift Closures

Closures

- Closures are anonymous functions (often referred to as lambdas or blocks in other languages). They are blocks of code that you can pass as arguments to functions where the receiving function expects a function type.
- In Swift, closures are a special type of functions that can be defined without the use of the *func* keyword and a function name.
- Just like functions, the closures can accept parameters and return values.
- A closure also has a set of statements that will be executed once the closure is called.
- Just like functions, a closure can be assigned to a variable or a constant.

Introduction

Closures Declaration

- In Swift, a closure is a special type of function without the function name. For example

```
{
  print("Hello World")
}
```

Swift Closure Declaration:

- We don't use the `func` keyword to create closure

```
{ (parameters) -> returnType in
   // statements
}
```

Introduction

Closures Declaration

Swift Closure Declaration:

- We don't use the `func` keyword to create closure

```
{ (parameters) -> returnType in
   // statements
}
```

Here,

1. **parameters** - any value passed to closure
2. **returnType** - specifies the type of value returned by the closure
3. **in** (optional) - used to separate **parameters**/**returnType** from closure body

Example

```
var greet = {
  print("Hello, World!")
}
```

Here, we have defined a closure and assigned it to the variable named greet. Statement enclosed inside the {} is the closure body.

To execute this closure, we need to call it. Here's how we can call the closure

```
// call the closure

  greet()

  //Output : Hello World
```

Example

```
let addTwoNumbers = { (a: Int, b: Int) -> Int in
  return a + b
}
let sum = addTwoNumbers(3, 5)
print(sum)
// Output: 8
```

Closure Parameters

```
// closure that accepts one parameter
let greetUser = { (name: String)  in
    print("Hey there, \(name).")
}

// closure call
greetUser("Delilah")
```

ot parameters. For example,

- Call of Closure

ure parameter with body.

greetUser("Delilah")

Closure That Returns Value

```
// closure definition
var findSquare = { (num: Int) -> (Int) in
  var square = num * num
  return square
}

// closure call
var result = findSquare(3)

print("Square:",result)
```

n it's **return type** and use the **return**

Closure That Returns Value

```
var findSquare = { (num: Int) -> (Int) in
  ...
  return square
}
```

- -> (Int) - represents the closure return value of Int type
- return square - return statement inside the closure

The returned value is stored in the result variable.

Closures as Function Parameter

```
// define a function
func grabLunch(search: () -> ()) {
  …
  // closure call
  search()
}
```

ccepts closure as its parameter.

los

- search() - call closure from the inside o

Now, to call this function, we need to pass a

```
// function call
grabLunch(search: {
  print("Alfredo's Pizza: 2 miles away")
})
```

Closure

Expressions:

There are several special forms that allow closures to be written :

- A closure an omit the types of its parameters, its return types, or both.
- A closure may omit names for its parameters. Its parameters are then implicitly named `$` followed by their positions: `$0, $1, $2` and so on.
- A closure that consists of only a single expression is understood to return the value of that expression.

Closure

Expressions:

- The following expressions are equivalent:

  - ```
    myFunction { (x: Int, y: Int) ->Int in
        return x + y

    }
    ```

  - ```
    myFunction { x, y in
      return x + y
    }
    ```

  - ```
    myFunction { return $0 + $1 }
    ```

  - ```
    myFunction { $0 + $1 }
    ```

Trailing Closure

```
// function definition
func grabLunch(message: String, search: ()->()) {
  ...
}
```

e as its last parameter,

a function body without mentioning the name of the parameter. For example,

Trailing Closure Example

```
func grabLunch(message: String, search: ()->()) {
   print(message)
   search()
}

// use of trailing closure
grabLunch(message:"Let's go out for lunch")  {
  print("Alfredo's Pizza: 2 miles away")
}
```

Autoclosure

```
func display(greet: @autoclosure () -> ()) {
  ...
}

// without using {}
display(greet: print("Hello World!"))
```

aces {}.For example,

oclosure keyword in function definition. For example,

o pass an expression as an argument to a function, which is automatically wrapped in a closure.

Autoclosure: Example

```
// define a function with automatic closure
func display(greet: @autoclosure () -> ()) {
 greet()
}

// pass closure without {}
display(greet: print("Hello World!"))
```

//Output: Hello World!